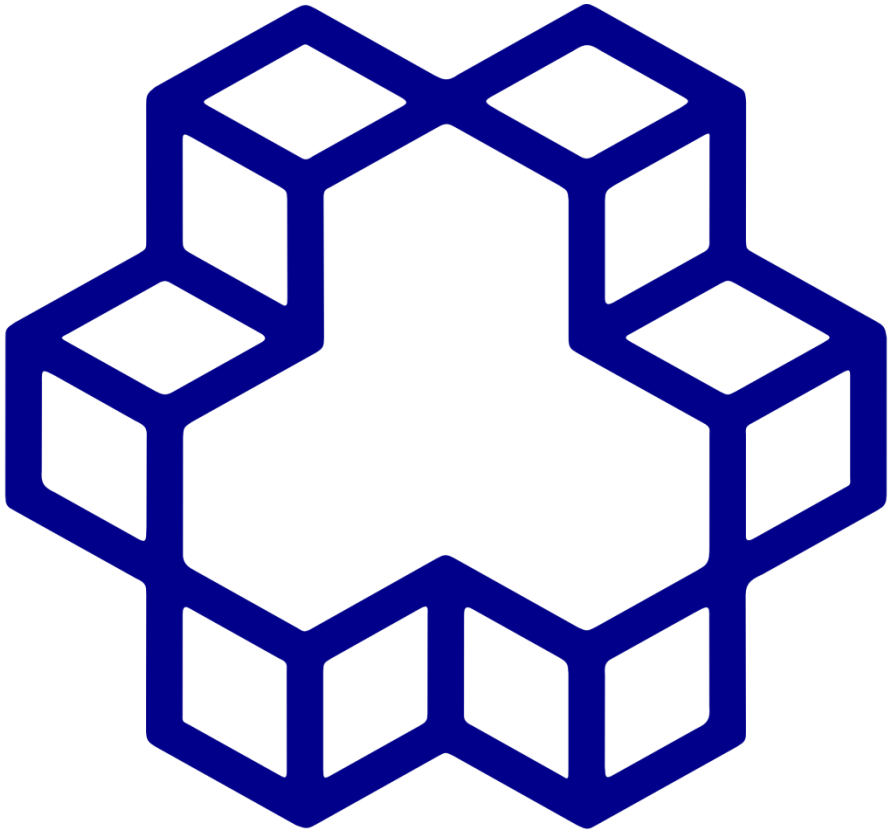


# دانشگاه صنعتی خواجه نصیرالدین طوسی

به نام خدا

ریزپردازنده و زبان اسمبلی

نام پروژه : بازی سرعت عمل ۲ نفره

نام تهیه کننده : محمد کاردان ، ۴۰۲۲۰۴۱۳

# مقدمه :

در این پروژه قصد داریم با پردازنده Atmega64 یک بازی سرعت عمل دو نفره طراحی کنیم که دو بازیکن در یک زمان مشخص باید یک الگوی باینری رو وارد کنن تا امتیاز بگیرن . قسمت امتیازی پروژه نیز زده شده است.

1. بازی در وضعیت اولیه منتظر دریافت دستور شروع است و LED سفید به صورت چشمک زن کار میکند.
2. پس از دریافت شروع یک کاربر به صورت رندوم انتخب شده و الگوی باینری به مدت ۵ ثانیه نمایش داده شده و در این ۵ ثانیه LED زرد چشمک زن کار میکنه بعد از این زمان این LED یک ثانیه روشن میماند بعد ۱۰ ثانیه نوبت حرکت کاربر شروع میشود .
3. در این زمان در صورت وارد کردن صحیح کد LED سبز چشمک زن میشود در صورت غلط وارد کردن LED قرمز چشمک زن میشود .
4. بعد نوبت کاربر بعدی میشود و بعد از اتمام راند اول امتیاز های هر دو نفر نمایش داده و LED برنده روشن شده .
5. بازی ۳ راند دارد و بعد از انجام بازی LED برنده روشن میشود .(در صورت تساوی هر دو LEDروشن شده)

نمایش کد باینری از طریق کد LED BARGRAPH متصل به PORTA انجام میشود .

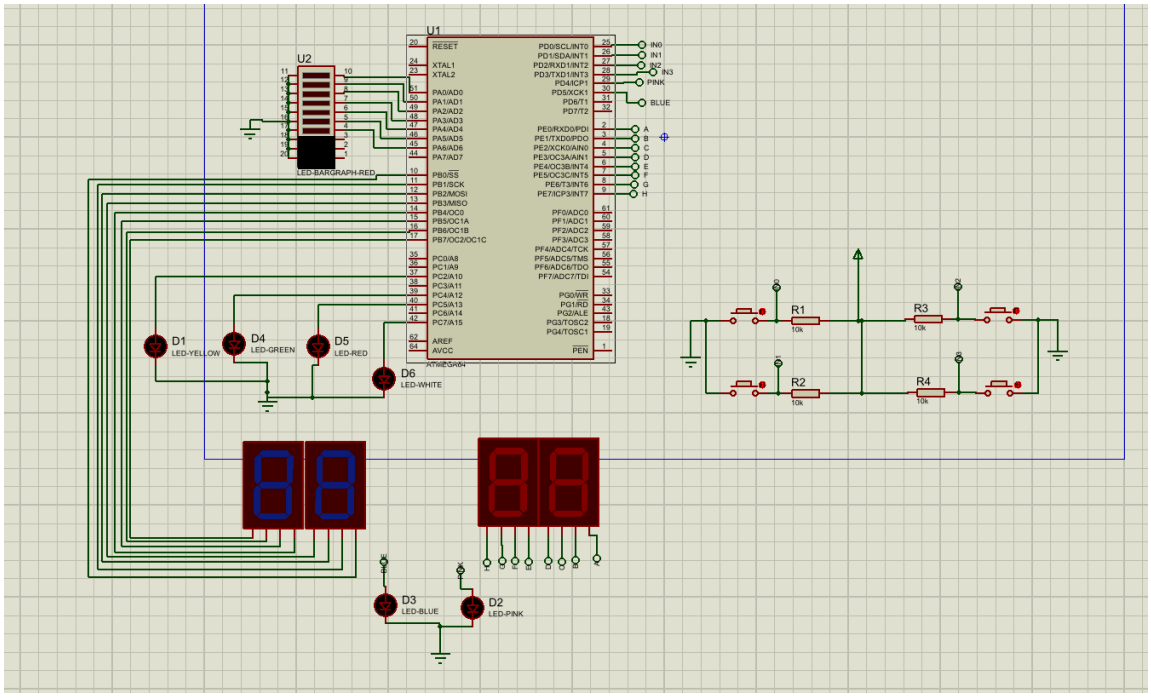
از وقفه های خارجی INT0 تا INT3 برای ورودی گرفتن از کاربر استفاده میشود .

نمایش امتیاز بازیکنان هم از طریق PORT.Bو PORT.E انجام میشود .

همچنین فرکانس پردازنده 1.024Mhz میباشد .

آماده سازی این پروژه در نرم افزار Atmel Studio انجام شده و در برنامه Proteus شبیه سازی شده .

در ادامه توضیحات کد اسمبلی قرار داده شده و **همچنین برای توضیح بهتر فیلم نیز اماده گردیده** .



```
        .Include "M64DEF.INC"

        .ORG    0X0000
        JMP     main
        .ORG    0x0002
        JMP     INT0_ISR
        .ORG    0x0004
        JMP     INT1_ISR
        .ORG    0x0006
        JMP     INT2_ISR
        .ORG    0x0008
        JMP     INT3_ISR
        .ORG    0x0020
        JMP     Timer0_ISR
        .ORG    0x0050
```

در این بخش کتابخانه خوانده شده و همچنین ادرس های وقفه ها و تایمر مشخص گردیده .

```
main:   LDI     R20,low(RAMEND)
        OUT     SPL,R20
        LDI     R20,high(RAMEND)
        OUT     SPH,R20                 ;SP = 0x10FF
        LDI     R20,0x0AA
        STS     EICRA,R20               ;FALLING EDGE INT1 AND INT2 AND INT3
        LDI     R20,8
        OUT     EIMSK,R20               ;INT3   ACCEPTED
        LDI     R20,1
        OUT     TIMSK , R20
        LDI     R20,0x06
        OUT     TCCR0,R20
        LDI     R20,56
        OUT     TCNT0, R20              ; timer pulse is 50ms and Fcpu is 1.024Mhz
        LDI     R20,0x0FF
        OUT     DDRA,R20
        OUT     DDRC,R20
        OUT     DDRB,R20
        OUT     DDRE,R20
        SBI     DDRD,4
        SBI     DDRD,5                  ;BOB LED
        CLR     R20


        LDI     R28,5
        LDI     R27,50
        SEI
```

اینتراپت ها به صورت پایین رونده فعال شده و در این مرحله صرفا INT3 مورد پذیرش میباشد

بعد برای تایمر و کلاک فرکانس به ۲۵۶ تقسیم شده و مقدار ریجستر TCNT0 ۵۶ شده تا با هر ۲۰۰ تا شمردن وقفه تایمر فعال بشود . در این صورت هر 50ms وقفه تایمر فعال شده

بعد پورت های A , C , B, E و همینطور D5, D4 , نیز خروجی شده اند

ریجستر R28 مقدارش ۵ شده که بیانگر تعداد اعداد باینری هر راند است که زیاد میشود . ریجستر R27 نیز برای تعیین برنده کل استفاده شده در صورت برد BOB یکی زیاد و برد ALICE یکی کم میشود و در پایان با مقدار اولیه مقایسه میشود.

```
loop:
        CPI     R31,1
        BREQ    start
        CPI     R18,10
        BRLO    loop
        CLR     R18
        LDI     R23,0x80
        IN      R24,PORTC
        EOR     R24,R23
        OUT     PORTC,R24
        JMP     loop
```

در اینجا برنامه منتظر دستور کاربر برای استارت بازی است که با زدن بر دکمه INT3 انجام میشود و در همین حین LED سفید متصل به PORTC.7 به صورت چشمک زن با دوره تناوب ۱ ثانیه و دیوتی سایکل ۵۰ درصد هست و در صورت خوردن وقفه برنامه به Start جامپ میکند و بازی شروع میشود.

```
INT3_ISR:   LDI     R31,1
            RETI
```

```
start:      CBI     PORTC,7
            CLR     R29
            ADD     R29,R28
            LDI     R20,0
            OUT     EIMSK,R20           ;INT3  NOT ACCEPTED
            IN      R16,TCNT0
            EOR     R16,R15
            MOV     R17,R16
            LDI     R22,1
loop1:      LSL     R22
            INC     R22
            DEC     R29
            BRNE    loop1
            LSR     R22
            AND     R17,R22             ;R17 IS THE RANDOM NUMBER
            SBRC    R16,7
            JMP     BOB
            JMP     ALICE
```

برای اطمینان LED سفید خاموش شده اینتراپت ۳ دیگه مورد پذیرش نخواهد بود مقدار عدد تصادفی از TCNT0 خوانده شده (علت اینکه فرکانس پردازنده به ۲۵۶ تقسیم شد این بود که مقدار TCNT0 بازه بیشتری باشد و احتمال دیدن عدد رندوم بازه اش گنده تر باشد) با ریجستر ۱۵ ایکسور میکنیم تا رندوم شود از تابع loop1 و ریجستر های R22 , R29 برای برداشتن تعداد بیت های لازم استفاده شده چون هر راند یک عدد اضافه میشه من از این روش استفاده کردم تا بیت های لازم رو استخراج کنم

بعد از مقایسه بیت ۷ ام عدد تصادفی برای تعیین نوبت استفاده میشود به این صورت که اگه نوبت اول با BOB باشد برنامه به BOB میپرد و سپس به ALICE و بعد راند تمام میشود و اگر نوبت با ALICE باشد اول به ALICE جامپ میزند و سپس به BOB و بعد راند تمام میشود.

```
BOB:        CLR     R31
            OUT     PORTA,R17
            SBI     PORTD,5
            CLR     R18
            CLR     R21
```

برنامه BOB و ALICE دقیق عین هم اند فقط اسم ها جابجا شده و خب ریجستر ذخیره امتیاز و LED متفاوتی دارند وگرنه کل کد یکیه . فرض بگیریم برنامه به BOB جامپ بزند .

```
waiterBOB:
            CPI     R18,100
            BREQ    NEXTBOB
            CPI     R21,5
            BRNE    waiterBOB
            CLR     R21
            LDI     R23,0x04
            IN      R24,PORTC
            EOR     R24,R23
            OUT     PORTC,R24
            JMP     waiterBOB
```

مقدار R31 که قبلا استفاده شده بود صفر شده تا در کد دوباره استفاده شود . به عنوان شمارنده تعداد بیت های داده شده توسط بازیکن عدد تصادفی روی PORTA نمایش داده شده LED ابی متصل به PORTD.5 روشن شده و این یعنی نوبت BOB شده ریجستر های R18و R21 در تایمر هر ۵۰ میلی ثانیه یکی زیاد میشوند . در اینجا برنامه به مدت ۵ ثانیه عدد تصادفی رو نشون میده و LED زرد چشمک زن میشود به دوره تناوب ۰.۵ ثانیه و دیوتی سایکل ۵۰ درصد . بعد از پنج ثانیه برنامه به NEXTBOB میپرد

حالا PORTA خاموش شده و LED زرد به مدت یک ثانیه روشن میماند

در WAITER2 R18 با ۲۰ مقایسه شده (1s=20*50ms)

بعد اینتراپت ها یا همان ورودی های کاربر قابل پذیرش میشوند

R30 که ورودی کاربر در آن ذخیره میشود صفر میشود تا ورودی بگیریم

برنامه به BOBTURN میرود و نوبت ۱۰ ثانیه بازیکن شروع میشود (10s=200*50ms)

هروقت تعداد بیت های ورودی کاربر به مقدار R28 برسد

برنامه CHECK رو کال میکنه تا مشخص شه عدد درست وارد شده یا نه

و در نهایت بعد از ۱۰ ثانیه نوبت . وقفه ها غیر قابل پذیرش شده و

LED بازیکن که اینجا آبی است خاموش شده و با مقایسه بیت ۷ ام عدد تصادفی

نوبت بازیکن بعدی تعیین میشود یا به ROUNDend میپرد

```
NEXTBOB:    CLR     R18
            OUT     PORTA,R18
            SBI     PORTC,2
WAITER2:    CPI     R18,20
            BRNE    WAITER2
            CBI     PORTC,2
            CLR     R18
            LDI     R20,7
            OUT     EIMSK,R20       ;INT 0,1,2 IS ACCEPTED
            CLR     R31
            CLR     R30
BOBTURN:    CBI     PORTC,2
            CPI     R18,200
            BRSH    BOBN
            CP      R31,R28
            BRNE    BACKER
            CALL    CHECK
BACKER:     JMP     BOBTURN

BOBN:       LDI     R20,0
            OUT     EIMSK,R20
            CBI     PORTD,5
            SBRS    R16,7
            JMP ROUNDend
            JMP ALICE
```

در اینجا برنامه ALICE اجرا میشود که دقیقا با BOB یک است

```
ALICE:      CLR       R31
            OUT    PORTA,R17
            SBI    PORTD,4
            CLR    R18
            CLR    R21

waiterALICE: CPI    R18,100
            BREQ   NEXTALICE

            CPI    R21,5
            BRNE   waiterALICE
            CLR    R21
            LDI    R23,0x04
            IN     R24,PORTC
            EOR    R24,R23
            OUT    PORTC,R24
            JMP    waiterALICE

NEXTALICE:  CLR       R18
            OUT       PORTA,R18
            SBI       PORTC,2
WAITER3:    CPI     R18,20
            BRNE    WAITER3
            CBI     PORTC,2
            CLR     R18
            LDI     R20,7
            OUT     EIMSK,R20           ;INT 0,1,2 IS ACCEPTED
            CLR     R31
            CLR     R18
            CLR     R30
ALICETURN:  CBI    PORTC,2
            CPI     R18,200
            BRSH    ALICEN
            CP      R31,R28
            BRNE    BACKER1
            CALL    CHECK
BACKER1:        JMP     ALICETURN

ALICEN:     LDI     R20,0
            OUT     EIMSK,R20
             CBI    PORTD,4
             SBRC   R16,7
             JMP    ROUNDend
             JMP    BOB
```

```
INT0_ISR:       PUSH    R20
                IN      R20, SREG
                PUSH    R20
                LSL     R30
                INC     R31
                INC     R15
                POP     R20
                OUT     SREG,R20
                POP     R20
                RETI

INT1_ISR:       PUSH    R20
                IN      R20, SREG
                PUSH    R20
                LSL     R30
                INC     R30
                INC     R31
                INC     R15
                POP     R20
                OUT     SREG,R20
                POP     R20
                RETI

INT2_ISR:       PUSH    R20
                IN      R20, SREG
                PUSH    R20
                CLR     R30
                CLR     R31
                POP     R20
                OUT     SREG,R20
                POP     R20
                RETI
```

منطق برنامه برای ورودی گرفتن کاربر به صورت زیر است .

در صورت وارد کردن INT0 یعنی عدد صفر اضافه شه که با دستور LSL

اینکارو کردیم و همینطور ریجستر شمارنده یکی زیاد میشود

در صورت وارد کردن INT1 یعنی کاربر عدد یک وارد کرده

پس عدد R30 یه بیت به چپ شیفت میخورد و به آن یک اضافه میشود

در کل این برنامه مقدار R20و SREG ذخیره شده چون دستورات INC , LSL

مقادیر فلگ را تغییر داده و در برنامه اختلال ایجاد میکند .

وظیفه INT2 هم ریست کردن وردی کاربر و همینطور شمارنده تعداد بیت های ورودی است

ریجستر ۱۵ مقدار کلیک های اینتراپت ها را میشمرد تا در تولید عدد تصادفی بعدی

استفاده شود

```
Timer0_ISR:     PUSH    R20
                IN      R20,SREG
                PUSH    R20
                LDI     R20,56
                OUT     TCNT0, R20
                INC     R18
                INC     R21
                POP     R20
                OUT     SREG,R20
                POP     R20
                RETI

                 RETI
```

در تایمر برنامه هم که با هر ۵۰ میلی ثانیه یکی به ریجستر های R18 , R21

اشافه میشود

```
CHECK:  CBI     PORTC,2
        LDI     R20, 0
        OUT     EIMSK, R20
        CP      R17, R30
        LDI     R22,8
        BREQ    IS_RIGHT
        RJMP    IS_WRONG
IS_RIGHT:   SBIC    PORTD, 5
            INC     R25
            SBIC    PORTD, 4
            INC     R26
            LDI     R23, 0x10
            RJMP    BLINKER_LOOP1


IS_WRONG:       LDI     R23, 0x20
```

وقتی داخل برنامه CHECK کال میشود یعنی کاربر تعداد بیت های کافی وارد کرده

پس چک میکنیم که درسته یا غلطه

اول دیگه پذیرش وقفه متوقف میشه . R22 برای روشن کردن LED سبز

یا قرمز به مدت ۸۰۰ میلی ثانیه است .

در صورت درست بودن ورودی کاربر با بررسی LED های هر بازیکن

امتیاز به همان بازیکن داده میشود و مقدار R23 0001000 میشود

تا با آن LED سبز چشمک زن شود و در صورت غلط بودن

0010000 وارد آن میشود تا LED قرمز چشمک بزند

```
BLINKER_LOOP1:      IN      R24, PORTC
                    EOR     R24, R23
                    OUT     PORTC, R24

                    RCALL   DELAY_100

                    DEC     R22
                    BRNE    BLINKER_LOOP1

                    CBI     PORTC, 2
                    CBI     PORTC, 4
                    CBI     PORTC, 5

        RJMP    CHECKEND


DELAY_100:
        PUSH    R23
        PUSH    R24

        LDI     R23, 200
Loop11:  LDI     R24, 127
Loop22:  NOP
        DEC     R24
        BRNE    Loop22
        DEC     R23
        BRNE    Loop11

        POP     R24
        POP     R23
        RET
CHECKEND:   LDI     R20,7
            OUT     EIMSK,R20
            CLR     R30
            CLR     R31
            RET
```

داخل BLINLER_LOOP برنامه به مدت ۸۰۰ میلی ثانیه

هر ۱۰۰ میلی ثانیه LED مورد نظر رو نات میکنه تا جشمک زن شود

این یعنی ۸ بار تغییر . من به علت مشکل تو استفاده از تایمر برای این بخش

برای ایجاد تاخیر ۱۰۰ میلی ثانیه از تاخیر نرم افزاری استفاده کردم

با کال کردن DELAY_100

این زیربرنامه ۱۰۲۴۱۲ کلاک پردازنده طول میکشه تا اجراشه

با فرکانس ما میشود ۱۰۰ میلی ثانیه

بعد از ۸ بار چشمک زدن مقدار R22 صفر شده و برنامه به CHECKEND میپرد

حالا دوباره اماده پذیرش وقفه و وردی کاربر هستیم

و همینطور ورودی کاربر و شمارنده را صفر میکنیم

```
ROUNDend:   OUT     PORTB,R25
            OUT     PORTE,R26
            LDI     R16,5
            INC     R28
            CP      R25,R26
            BREQ    DRAW
            BRMI    AWIN
            JMP     BWIN
```

در صورت اتمام راند به ROUNDEND میرسیم

امتیاز دو کاربر در پورت ها نمایش داده شده

ریجستر R16 پنج شده تا پنج ثانیه برنده به همراه امتیاز های هر دو بازیکن

نمایش داده شود

مقدار R28 یکی زیاد شده تا در راند بعدی مسابقه سختتر شود ( قسمت امتیازی )

امتیاز دو بازیکن مقایسه شده و برنامه به قسمت مورد نظرش جامپ میزند . در صورت برد BOB به BWIN

در صورت برد ALICE به AWIN و در صورت تساوی به DRAW میپرد

```
DRAW:
        CLR     R18
        SBI     PORTD,5
        SBI     PORTD,4
DDD:    CPI     R18,16
        BRLO    DDD
        CBI     PORTD,5
        CBI     PORTD,4
DDDD:   CPI     R18,20
        BRLO    DDDD
        DEC     R16
        BREQ    ER
        JMP     DRAW
ER:
            CLR     R25
            CLR     R26
            OUT     PORTB,R25
            OUT     PORTE,R26
            JMP     ENDD
AWIN:
        CLR     R18
        SBI     PORTD,4
AAA:    CPI     R18,16
        BRLO    AAA
        CBI     PORTD,4
AAAA:   CPI     R18,20
        BRLO    AAAA
        DEC     R16
        BREQ    ALENDER
        JMP     AWIN
ALENDER:    DEC     R27
            CLR     R25
            CLR     R26
            OUT     PORTB,R25
            OUT     PORTE,R26
            JMP     ENDD

BWIN:
        CLR     R18
        SBI     PORTD,5
BBB:    CPI     R18,16
        BRLO    BBB
        CBI     PORTD,5
BBBB:   CPI     R18,20
        BRLO    BBBB
        DEC     R16
        BREQ    BOBENDER
        JMP     BWIN
BOBENDER:   INC     R27
            CLR     R25
            CLR     R26
            OUT     PORTB,R25
            OUT     PORTE,R26
            JMP     ENDD

ENDD:

            CPI     R28,8
            BREQ    FINISH
            JMP     start
```

داخل DRAW ، LED هر دو بازیکن روشن شده

و با مقایسه R18 با ۱۶ . ۰.۸ ثانیه مدت زمان روشن بودنش رو میگذرونیم

بعد LED هارو خاموش کرده و ۰.۲ ثانیه دیگر صبر کرده و بعد برمیگردیم به لوپ

و مقدارشون رو یک میکنیم تا روشن شن

این عمل ۵ بار تکرار میشه تا مدت زمان کل خروجی هر راند ۵ ثانیه بشه

بعد پورت های خروجی B و E رو صفر کرده و به ENDD چامپ میکنیم

داخل AWIN هم عین DRAW اما صرفا بیت LED صورتی روشن خاموش میشخ

و مقدار R27 که نمایانگر برنده نهایی بود یکی کم میشه

داخل BWIN هم به همون صورت عمل میکنیم و مقدار R27 رو یکی زیاد میکنیم

در اخر مقایسه میکنیم R28رو با ۸ اگه به ۸ برسه یعنی ۳ راند بازی انجام شده

پس میریم به FINISH تا برنده نهایی نمایش داده شه و بازی تمام شه

اگه هم نه که برگرده به start تا راند بعد شروع شه

```
FINISH:     CPI     R27,50
            BREQ    MOSAVI
            BRSH    BOB_BORD
            SBI     PORTD,4
            JMP     TAMAM
MOSAVI:     SBI     PORTD,5
            SBI     PORTD,4
            JMP     TAMAM
BOB_BORD:   SBI     PORTD,5


TAMAM:      JMP     TAMAM
```

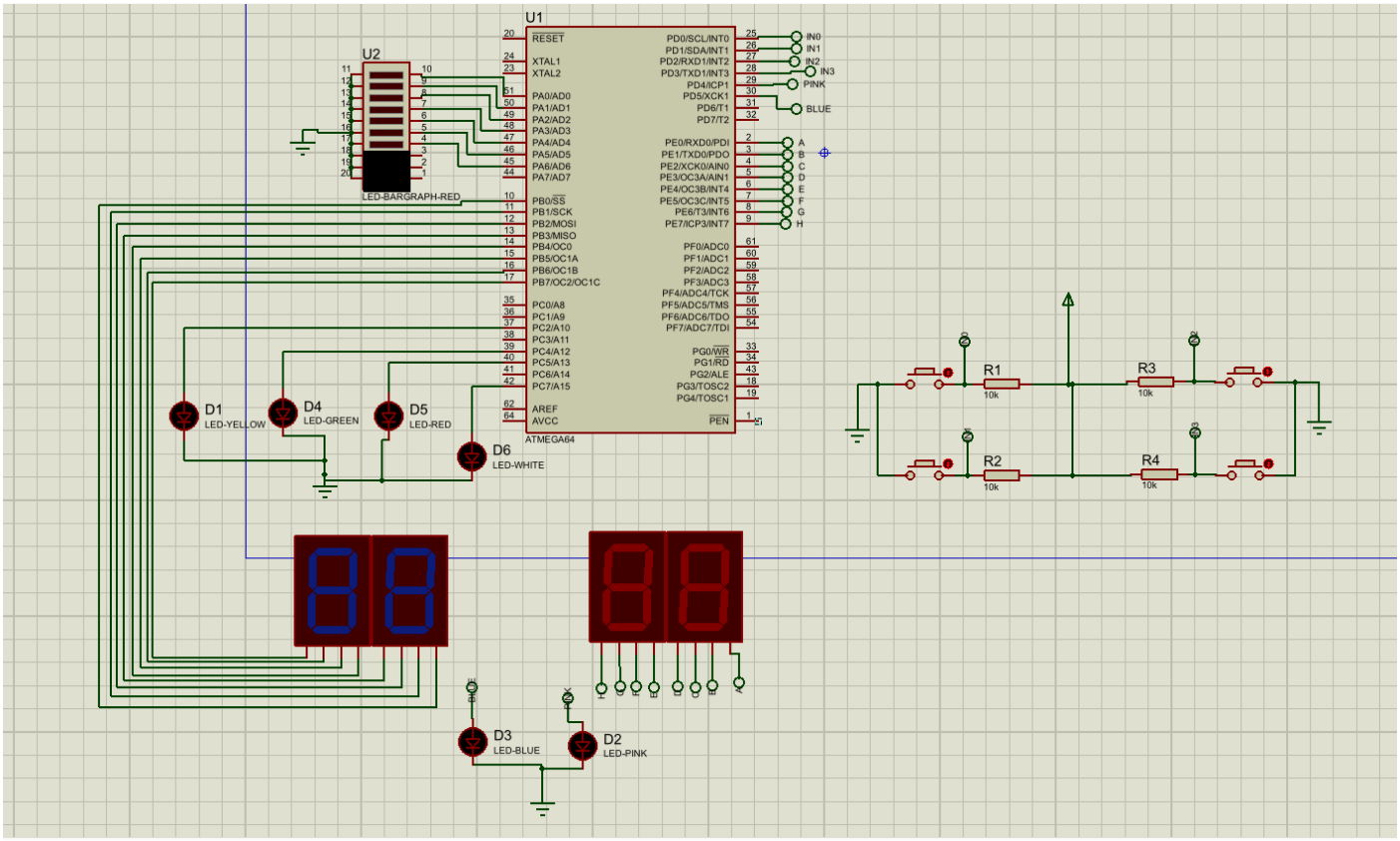
اینجا با مقایسه کردن R27 با ۵۰ که مقدار اولیه اون بود

میفهمیم برنده نهایی کیه

و LED برنده رو روشن میکنیم

در صورت تساوی هر دو LED روشن خواهد شد

و بازی به اتمام میرسد




# با تشکر از همراهی و توجه شما